



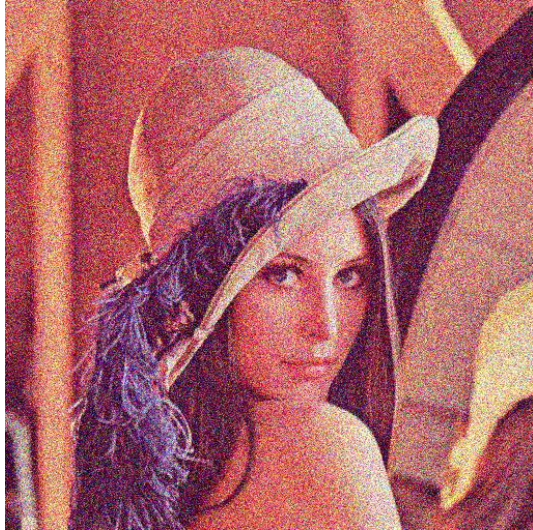
Image Denoising

Chao-Tsung Huang

**National Tsing Hua University
Department of Electrical Engineering**

Most fundamental image restoration problem

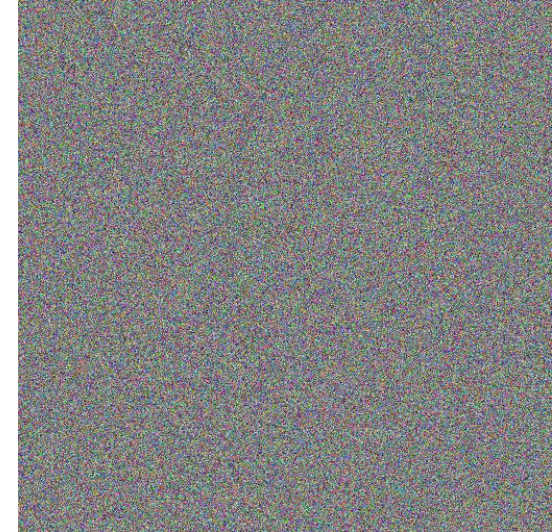
Noisy image



Latent image



Noise



$$\mathbf{y} = \mathbf{z} + \mathbf{n}$$


Denoise: Inverse problem to solve latent $\mathbf{z}$ from given observed $\mathbf{y}$
with assumptions on $\mathbf{z}$ and noise $\mathbf{n}$



Outline

- **Noise model for imaging**
- Categories of denoising algorithms
 - Local filter
 - Transform-based filter
 - Non-local filter
 - Sparsity-based filter



Image noise

- **Shot noise** (major noise source)

- Poisson distribution for captured photons

- $P(X = x | \mu_X = \lambda) = \frac{x^\lambda e^{-\lambda}}{\lambda!}$

- Critical for photon-limited imaging (astronomy, MRI, microscopy, etc.)

- Fixed pattern noise

- Thermal noise (Gaussian)

- ADC (quantization) noise (Uniform)

- Flicker noise (1/f)

} Electronic noise

Q: Why is additive white Gaussian noise (AWGN) often assumed?



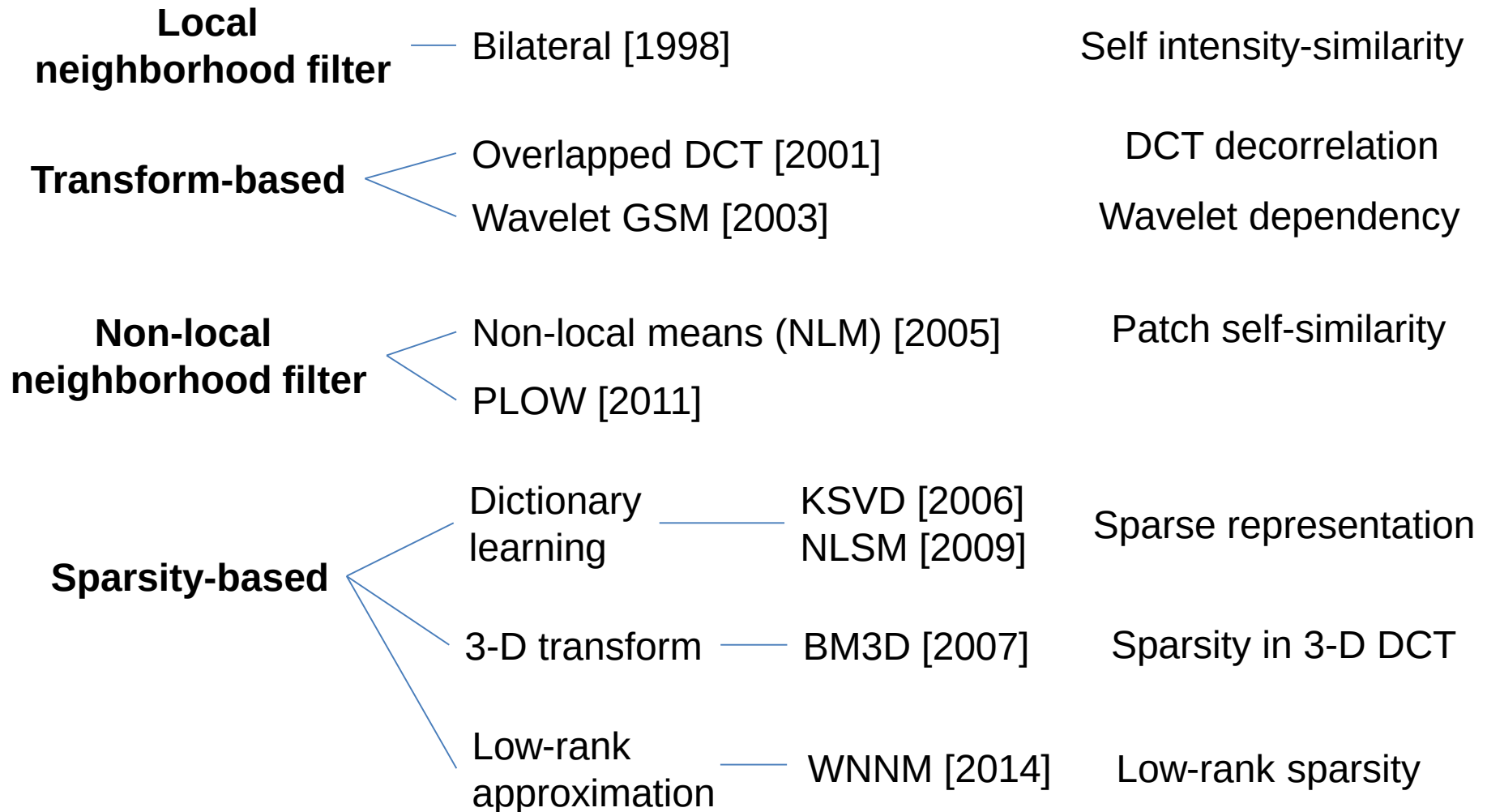
Why Gaussian?

- Analytically tractable
- Central limit theorem
- Coincidence between
 - Anscombe transform
 - Suppose X is Poisson distributed and $E(X) > 10$
 - $Y = \sqrt{X + 3/8} \Rightarrow Y \sim N(\sqrt{E(X) + \frac{3}{8}}, \frac{1}{4})$
 - Turn signal-dependent Poisson noise to signal-independent fixed-variance Gaussian!!
 - Gamma correction
 - sRGB or BT.709: $E \cong P^{0.5}$

So, shot noise in linear domain like Gaussian in non-linear domain.



Categories and selected algorithms for denoising





Outline

- Noise model for imaging
- Categories of denoising algorithms
 - **Local filter**
 - **Median filter**
 - **Bilateral filter**
 - Transform-based filter
 - Non-local filter
 - Sparsity-based filter

Non-linear filtering (Chap 3.3.1)

- For images, non-linear filtering often offers better quality

Gaussian Noise



Gaussian Filter
(linear)



Median Filter
(non-linear)



Bilateral Filter
(non-linear)



Salt&Pepper Noise



Median filter

- Select median value from neighborhood
 - Pro: Good to filter out bad pixels
 - Con: Not efficient to filter Gaussian noise; complexity
- Variants
 - α -trimmed mean (mean between α - and $(1-\alpha)$ -percentiles)
 - Weighted median

1	2	1	2	4
2	1	3	5	8
1	3	7	6	9
3	4	8	6	7
4	5	7	8	9

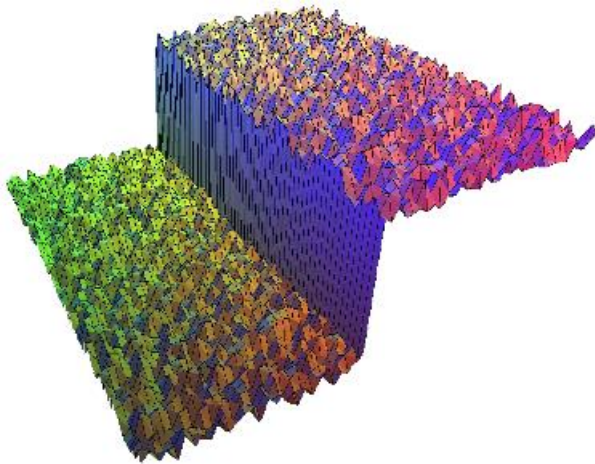
median = 4

1	2	1	2	4
2	1	3	5	8
1	3	7	6	9
3	4	8	6	7
4	5	7	8	9

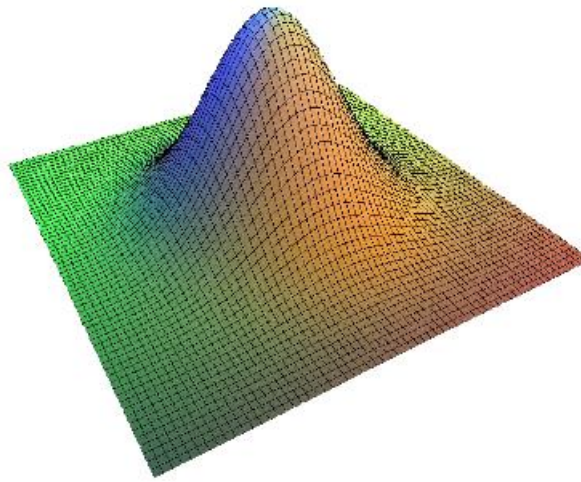
α -mean = 4.6

Bilateral filter

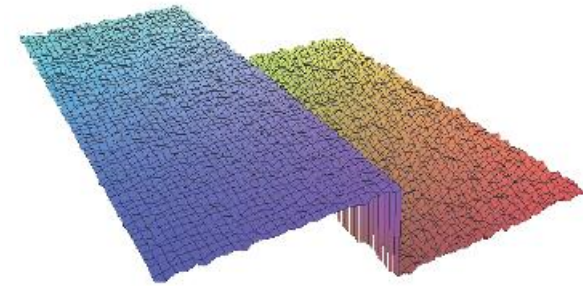
- Bilateral: domain and range kernels



Noisy edge input



Domain kernel (Gaussian)

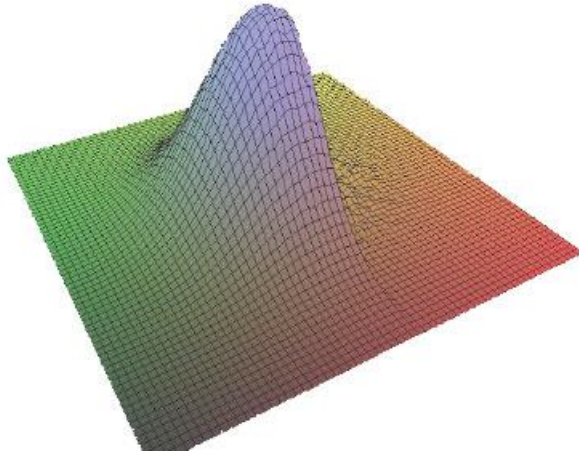


Range kernel

$$d(i, j, k, l) = \exp \left(-\frac{(i - k)^2 + (j - l)^2}{2\sigma_d^2} \right)$$

$$r(i, j, k, l) = \exp \left(-\frac{\|f(i, j) - f(k, l)\|^2}{2\sigma_r^2} \right)$$

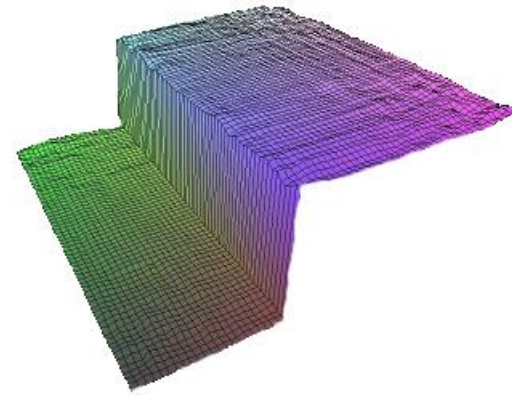
Bilateral filter



Bilateral kernel

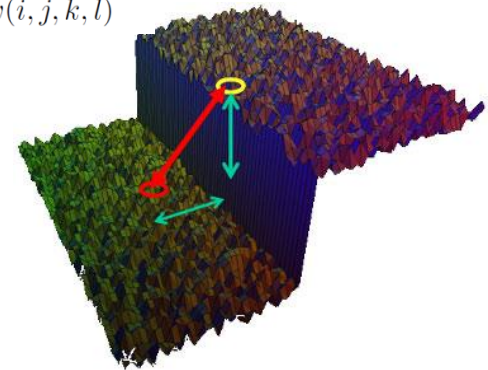
$$w(i, j, k, l) = \exp \left(-\frac{(i-k)^2 + (j-l)^2}{2\sigma_d^2} - \frac{\|f(i, j) - f(k, l)\|^2}{2\sigma_r^2} \right)$$

(= domain kernel x range kernel)



Filtered result (edge-preserved)

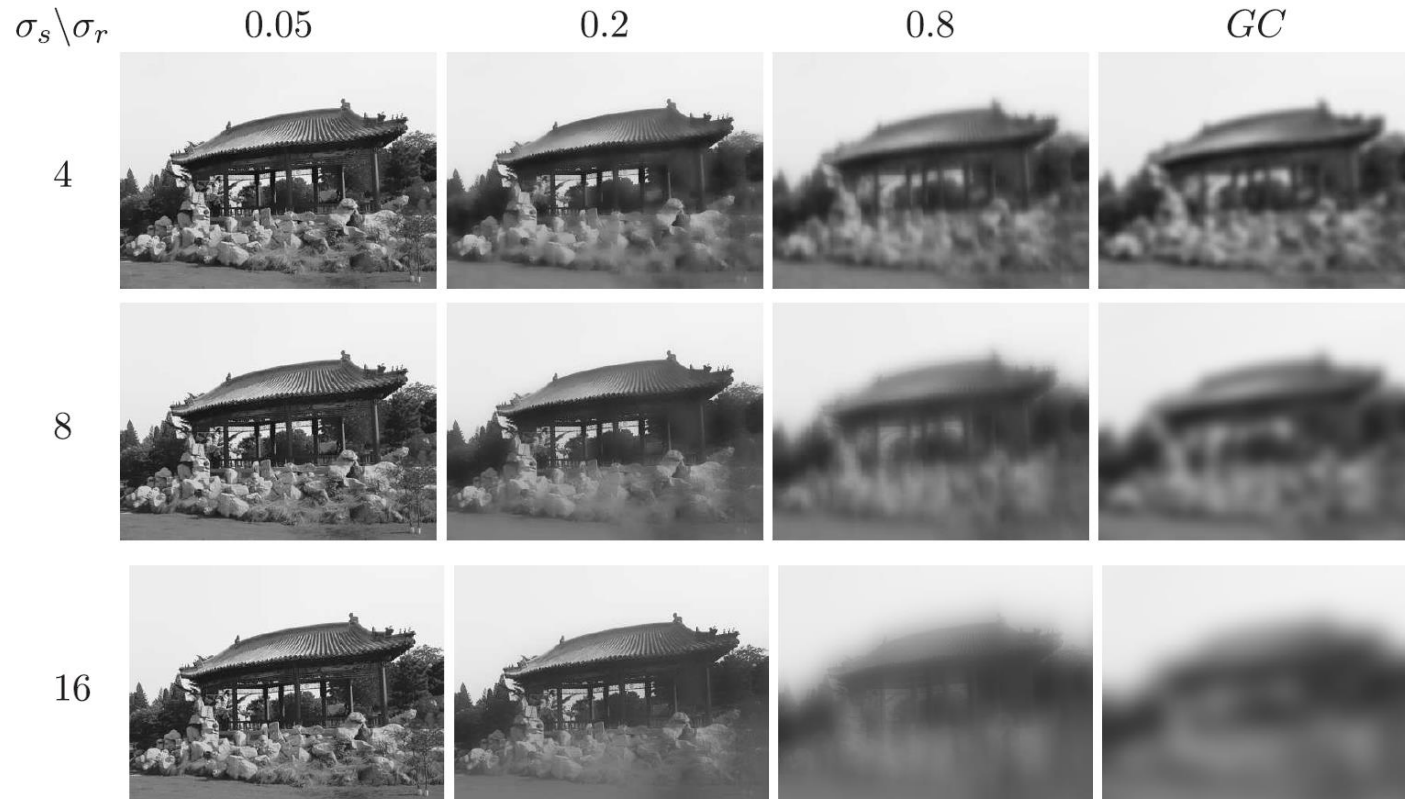
$$g(i, j) = \frac{\sum_{k,l} f(k, l) w(i, j, k, l)}{\sum_{k,l} w(i, j, k, l)}$$



More about bilateral filter

Effect of spatial and range parameters

(spatial kernel = domain kernel)



$$BF[I]_{\mathbf{p}} = \frac{1}{W_{\mathbf{p}}} \sum_{\mathbf{q} \in \mathcal{S}} G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|) G_{\sigma_r}(|I_{\mathbf{p}} - I_{\mathbf{q}}|) I_{\mathbf{q}} \quad W_{\mathbf{p}} = \sum_{\mathbf{q} \in \mathcal{S}} G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|) G_{\sigma_r}(|I_{\mathbf{p}} - I_{\mathbf{q}}|)$$

Ref : S. Paris, et. al., "Bilateral Filtering: Theory and Applications," in *Found. and Trends in Comput. Graph. and Vis.*, vol. 4, no. 1, 2008.

Applications

- Denoising
 - Edge-preserved filtering
- Contrast enhancement
 - Tone mapping, detail enhancement, ...
- Intensity-guided filter
 - Depth refinement
- Data fusion
 - Flash/non-flash imaging, ...



(a) Photograph with flash



(b) Photograph without flash



(c) Combination



Outline

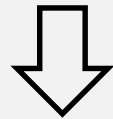
- Noise model for imaging
- Categories of denoising algorithms
 - Local filter
 - Transform-based filter
 - **Overlapped DCT**
 - Non-local filter
 - Sparsity-based filter



Prep: Pixel-wise Wiener filter

Additive Gaussian Model

$$Y = X + W, X \sim N(0, \sigma_x^2), W \sim N(0, \sigma_n^2)$$



for minimizing mean squared error

$$\hat{X} = E[X|Y] = \frac{\sigma_x^2}{\sigma_x^2 + \sigma_n^2} \cdot Y = \frac{1}{1 + \frac{1}{\text{SNR}}} \cdot Y$$

Higher SNR prefers more Y;
Otherwise, denoise more.

Implementation issue: How to estimate source variance σ_x^2 and noise variance σ_n^2 ?



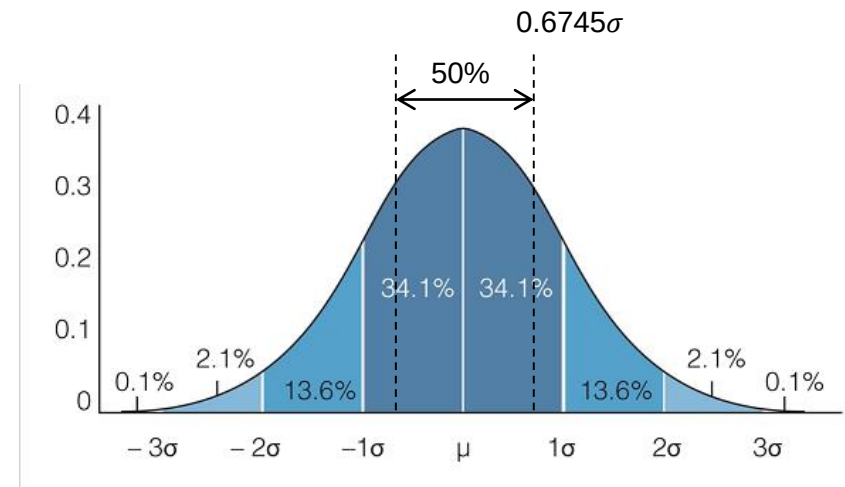
Prep: Median absolute deviation (MAD) for noise estimation

- MAD as a robust statistic
 - Don't need mean or variance
 - Robust when there are outliers
 - Given data set $X_1, X_2, \dots, X_n$,

$$MAD = \text{median}_i \text{abs}(X_i - \text{median}_j X_j)$$

- For Gaussian variable,

$$MAD = 0.6745 \cdot \sigma$$
$$\Rightarrow \sigma = 1.4826 \cdot MAD$$

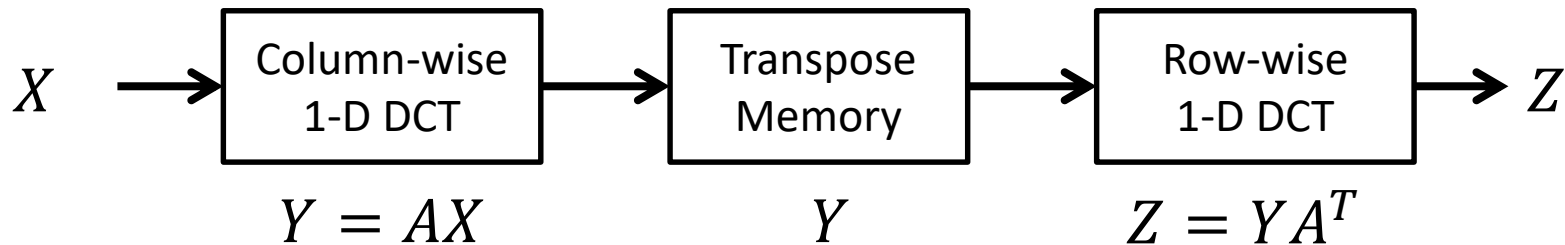




Prep: Discrete Cosine Transform (DCT)

- DCT on real-number signals
= DFT on real-number and mirrored signals
- Purpose
 - Frequency decomposition
 - **De-correlation**
- Separable 2-D requires $2N^3$ mul. $\Rightarrow O(N)$ per pixel
 - Direct 1-D N -point DCT: N^2 mul.

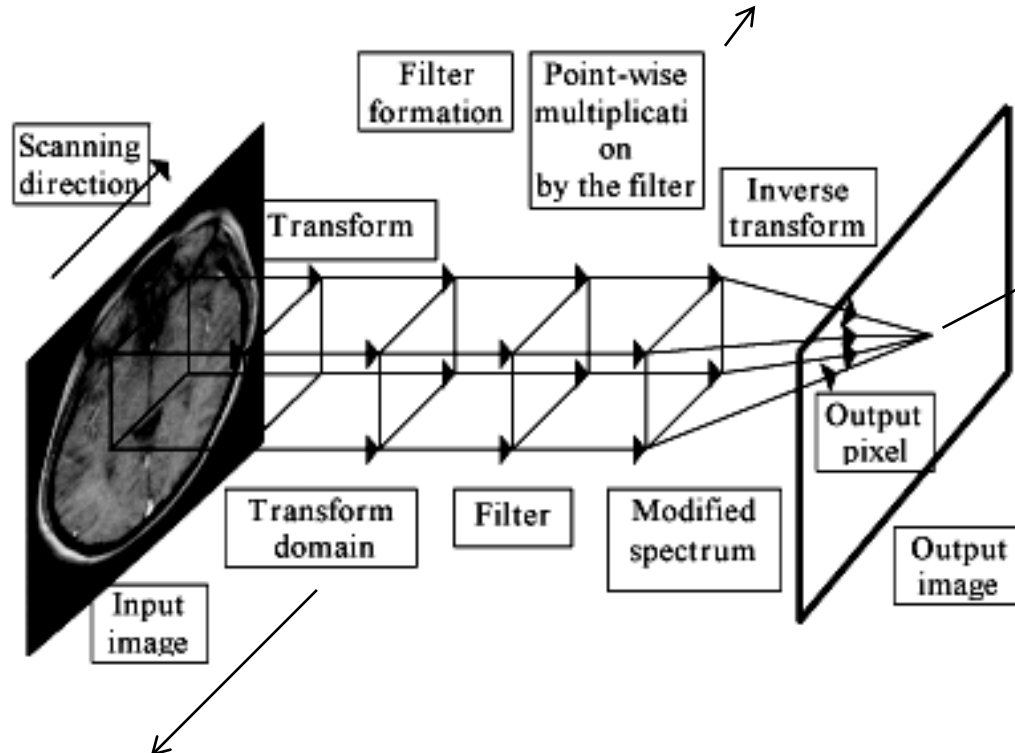
$$Z = AXA^T \quad A_{ij} = \sqrt{\frac{2}{N}} C(i) \cos \frac{(2j+1)i\pi}{2N}$$



Sliding-window DCT

Empirical Wiener filter:

Use transform coefficient as signal standard deviation σ_x



Reconstruction:

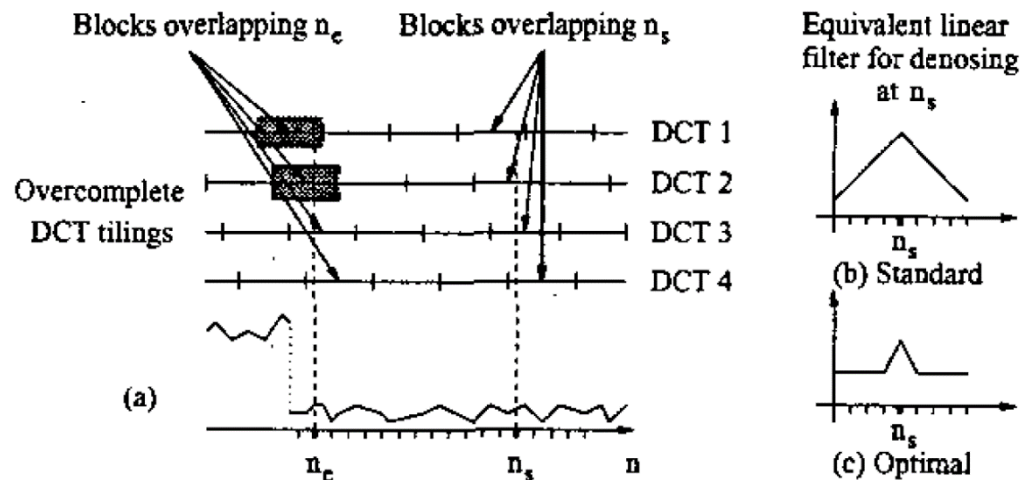
Each window estimates its central pixel

Decorrelation:

Make transform coefficients decorrelated for independent processing

Weighted overcomplete DCT denoising

- One pixel from one block is not reliable, but direct averaging will be blurry



Example of DC reconstruction only

$$\hat{x}_i = \mathbf{H}_i^{-1} \mathbf{S}_i \mathbf{H}_i y$$

$\nwarrow$ DCT $\downarrow$ Hard-thresholding (can be Wiener filter) $\nearrow$ DCT

$$\hat{x}(n) = \sum_{i=1}^M \alpha_i(n) \hat{x}_i(n), \quad n = 1, \dots, N$$

$\nwarrow$ Best weight should be inversely proportional to the energy



Outline

- Noise model for imaging
- Categories of denoising algorithms
 - Local filter: neighborhood histogram as PMF
 - Transform: statistical processing
 - **Non-local filter: patch self-similarity**
 - Non-local means
 - Sparsity-based filter

Non-local means (NLM)

- Still a weighted average filter
 - The weights calculated by non-local patch similarity
 - In contrast, bilateral filter uses local pixel similarity



Signal variance $\sigma_z = 3025$

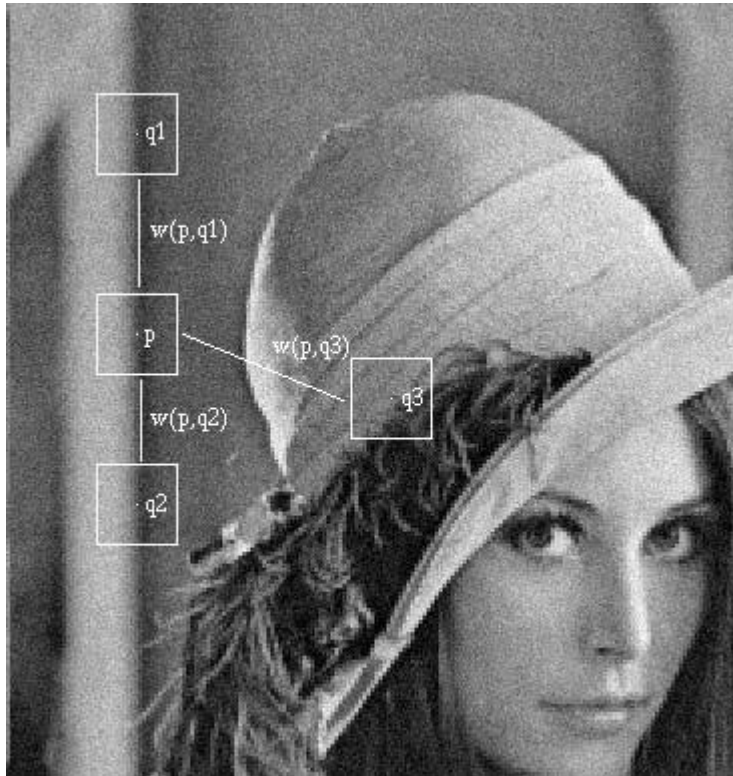


Noise variance $\sigma_n = 9$
(noise is invisible)



Noise variance $\sigma_n = 625$
(signal is still patch-wise visible)

Patch similarity



$$\mathbf{E} \left[\sum_{b \in \mathcal{B}} \| \mathbf{y}_{p+b} - \mathbf{y}_{q+b} \|^2 \right]$$

$$= \mathbf{E} \left[\sum_{b \in \mathcal{B}} \| \mathbf{z}_{p+b} - \mathbf{z}_{q+b} \|^2 \right] + 2B\sigma_n^2$$

($\mathcal{B}$: Set of patch pixels; $B = |\mathcal{B}|$)

l^2 -distance is a good indicator of patch similarity

 **NLM**

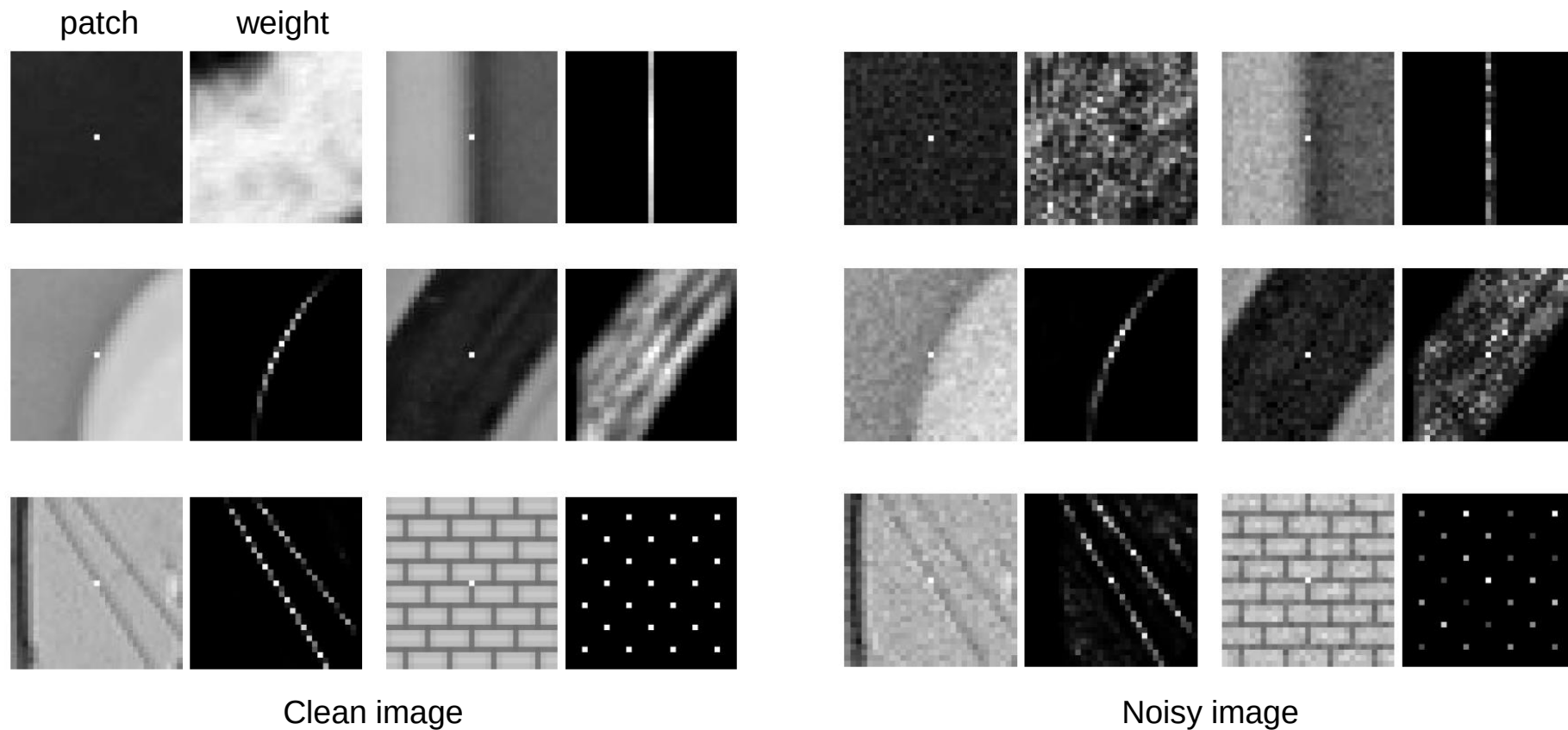
$$\mathbf{z}_l = \frac{\sum_{i \in \Lambda_l} w_{l,i} \cdot \mathbf{y}_i}{\sum_{i \in \Lambda_l} w_{l,i}}$$

Weighted average

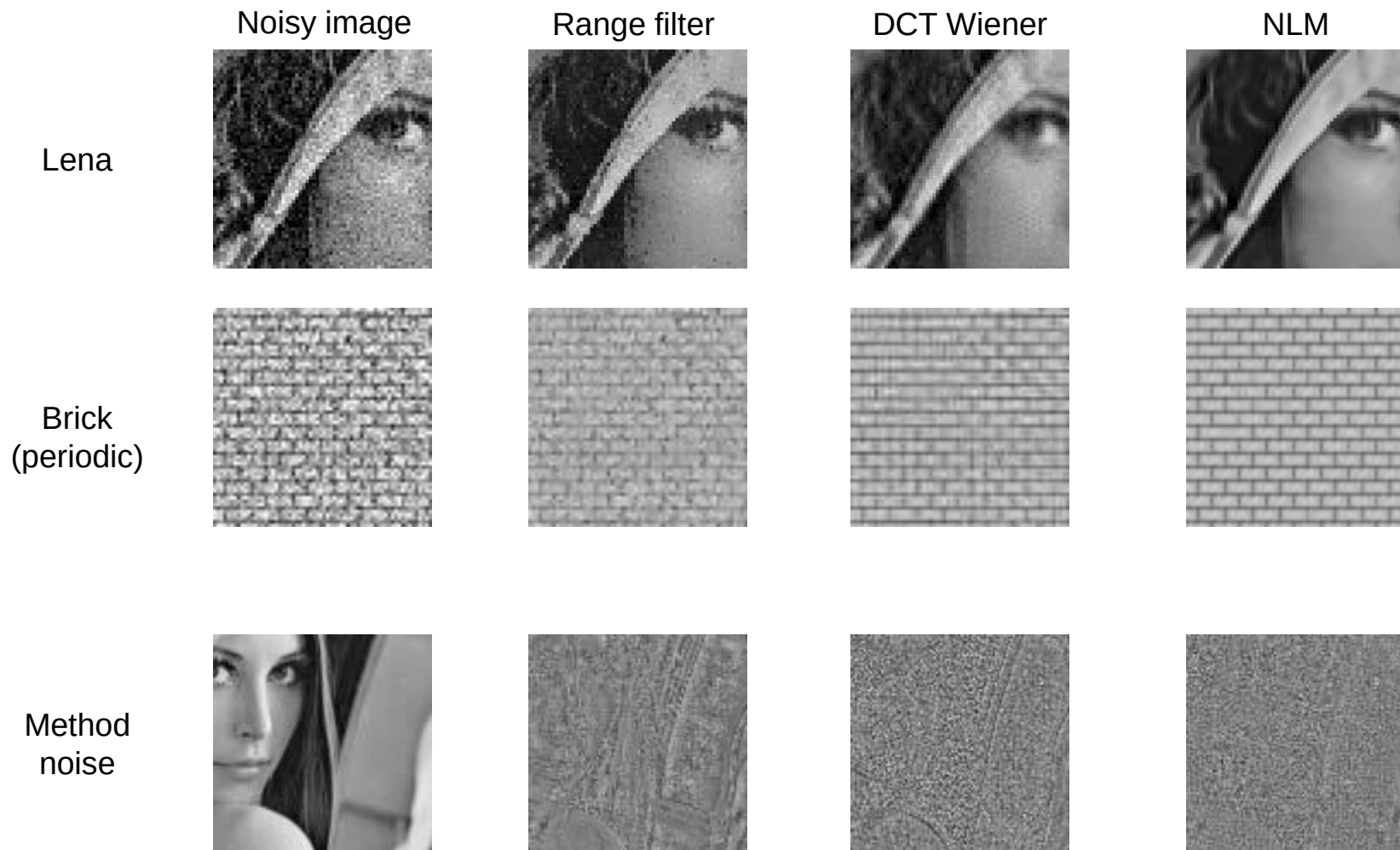
$$w_{l,i} = \exp\left(-\frac{\sum_{b \in \mathcal{B}} \| \mathbf{y}_{l+b} - \mathbf{y}_{i+b} \|^2_2}{2B\sigma_r^2}\right)$$

Patch weight

Patch Weight



Filtered results



The difference between the filtered $\hat{z}$ and the original z . **White** noise is preferred.



Issues and practical solutions

- Neighbor candidates
 - Issue: Too many non-similar candidates
 - Will blur the image and cause smaller σ_r^2 and search window
 - Solution: Select only the most similar candidates (motion estimation)
- Overlapped patches
 - Issue: Overlapped patches contribute to the same pixel
 - Solution: Aggregation using patch activity (inversely proportional)
- Selection of σ_r^2 value
 - Issue: Best value is image-dependent
 - Solution
 - Using Stein's unbiased risk estimate (**SURE**) to pick up the value; but incompatible to the non-differentiable motion estimation
 - Using Bayesian inference for neighborhood filters [C.-T. Huang, CVPR'15]



Outline

- Noise model for imaging
- Categories of denoising algorithms
 - Local filter
 - Transform-based filter
 - Non-local filter
 - **Sparsity-based filter**
 - Dictionary learning: K-SVD, NLSM
 - 3D transform: BM3D
 - Low-rank approximation: WNNM



Prep: Sparse representation

- Idea: Natural image can be sparsely expressed
 - A $\sqrt{n} \times \sqrt{n}$ patch (n pixels) can be decomposed into m vectors (atoms) where $m \ll n$
- Represent natural image patch y using sparse atoms from an overcomplete dictionary D

$$\hat{\alpha} = \arg \min_{\alpha} \|\alpha\|_0 \text{ subject to } \|D\alpha - y\|_2^2 \leq T$$



Relaxed as an unconstrained problem

$$\hat{\alpha} = \arg \min_{\alpha} \|D\alpha - y\|_2^2 + \mu \|\alpha\|_0$$

(can be implemented by
orthonormal matching pursuit)

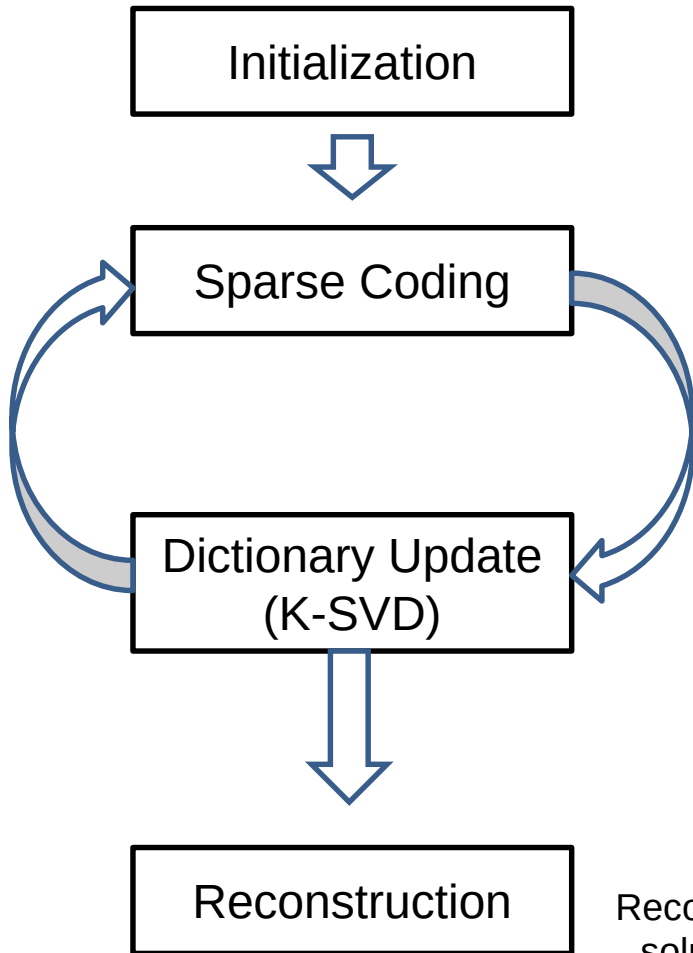
Dictionary (size $n \times k$, $k \gg n$)
(e.g. overcomplete DCT)

Atom weight vector
(size $k \times 1$)

Number of non-zero weights



K-SVD denoising



Dictionary D = Overcomplete DCT ($n=8 \times 8$, $k=256$)

Get sparse representation for each sliding-window patch

$$\hat{\alpha}_{ij} = \arg \min_{\alpha} \mu_{ij} \|\alpha\|_0 + \|\mathbf{D}\alpha - \mathbf{x}_{ij}\|_2^2$$

$\mathbf{x}_{ij} = \mathbf{R}_{ij} \mathbf{X}$
 i, j : position
 $\mathbf{X}$: whole image
 $\mathbf{R}_{ij}$: slicing matrix

Update the k dictionary atoms one-by-one (l -th atom):

1. Compute representation error without it for every patch using it

$$\mathbf{e}_{ij}^l = \mathbf{R}_{ij} \mathbf{X}_{ij} - \sum_{m \neq l} \mathbf{d}_m \alpha_{ij}(m)$$

2. Get overall error matrix $\mathbf{E}_l$ with those error column vectors
3. Update the atom using the most significant component by SVD

$$\mathbf{E}_l = \mathbf{U} \Delta \mathbf{V}^T$$

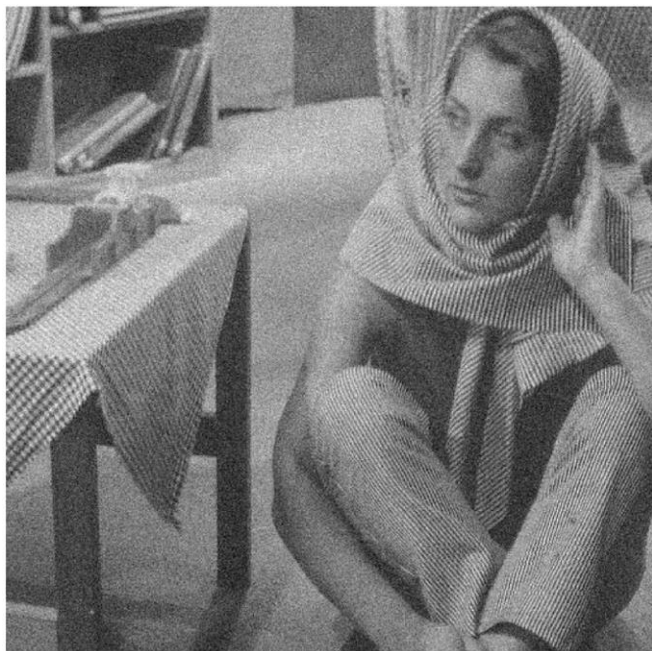
Reconstruct $\mathbf{X}$ by the close-form solution of the quadratic term

$$\mathbf{X} = \left(\lambda \mathbf{I} + \sum_{ij} \mathbf{R}_{ij}^T \mathbf{R}_{ij} \right)^{-1} \left(\lambda \mathbf{Y} + \sum_{ij} \mathbf{R}_{ij}^T \mathbf{D} \alpha_{ij} \right)$$

$$\hat{\mathbf{X}} = \arg \min_{\mathbf{x}} \lambda \|\mathbf{X} - \mathbf{Y}\|_2^2 + \sum_{ij} \|\mathbf{D} \hat{\alpha}_{ij} - \mathbf{R}_{ij} \mathbf{X}\|_2^2$$

Ref : M. Elad, et. al., "Image denoising via sparse and redundant representations over learned dictionaries," IEEE Trans. Image Process., 2006.

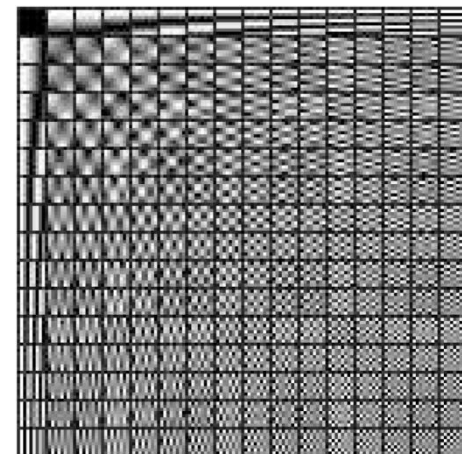
Denoising result



Barbara
($\sigma = 20$)



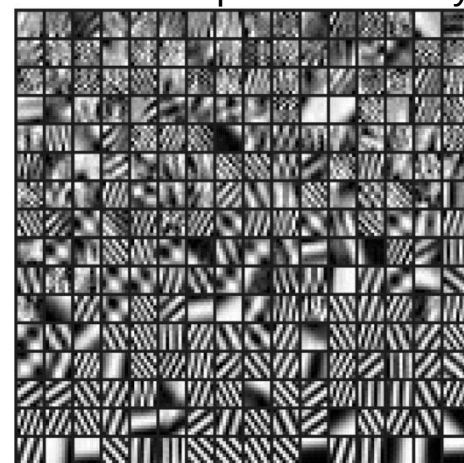
K-SVD
(MSE=54)



Overcomplete DCT

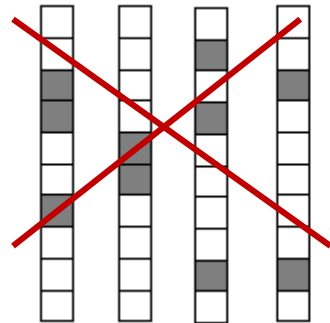


Trained adaptive Dictionary

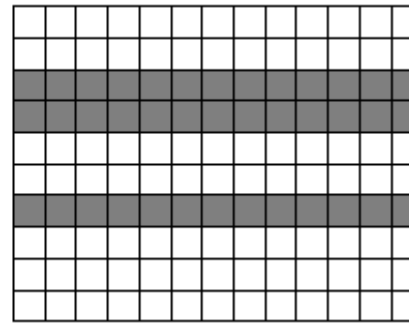


Non-local sparse model (NLSM)

- Improve K-SVD mainly in two points
 - For each patch, gather its similar non-local neighbors
 - Perform simultaneous sparse coding for the whole neighbor set
 - Pixel reconstructed by averaging all the estimates



Neighbor-independent sparsity



Joint sparsity

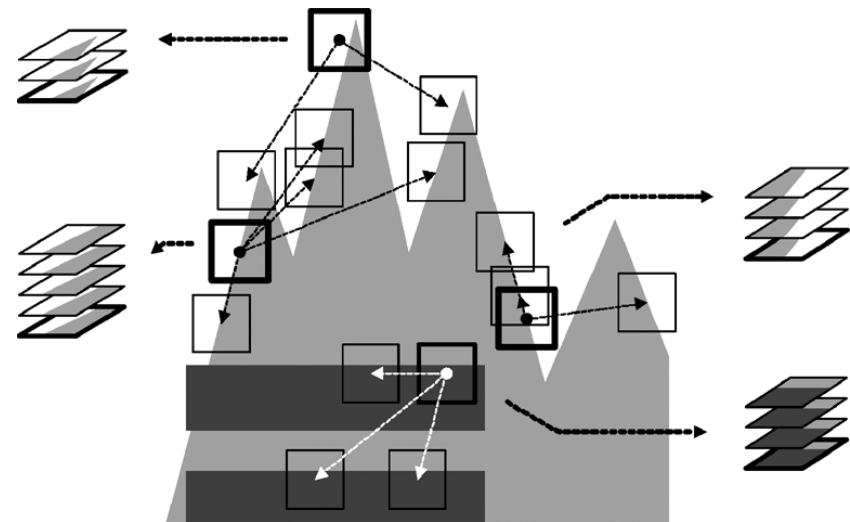
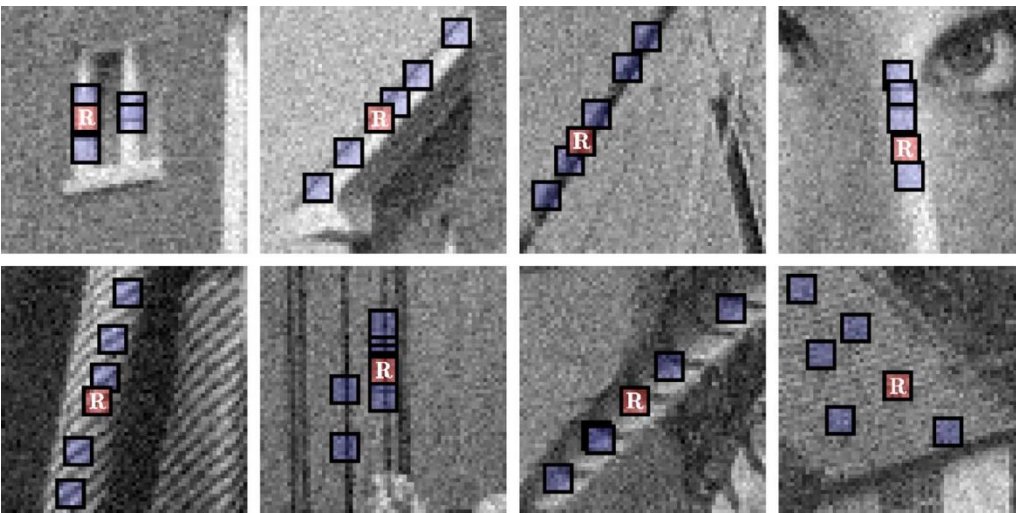


Block-matching and 3-D filtering (BM3D)

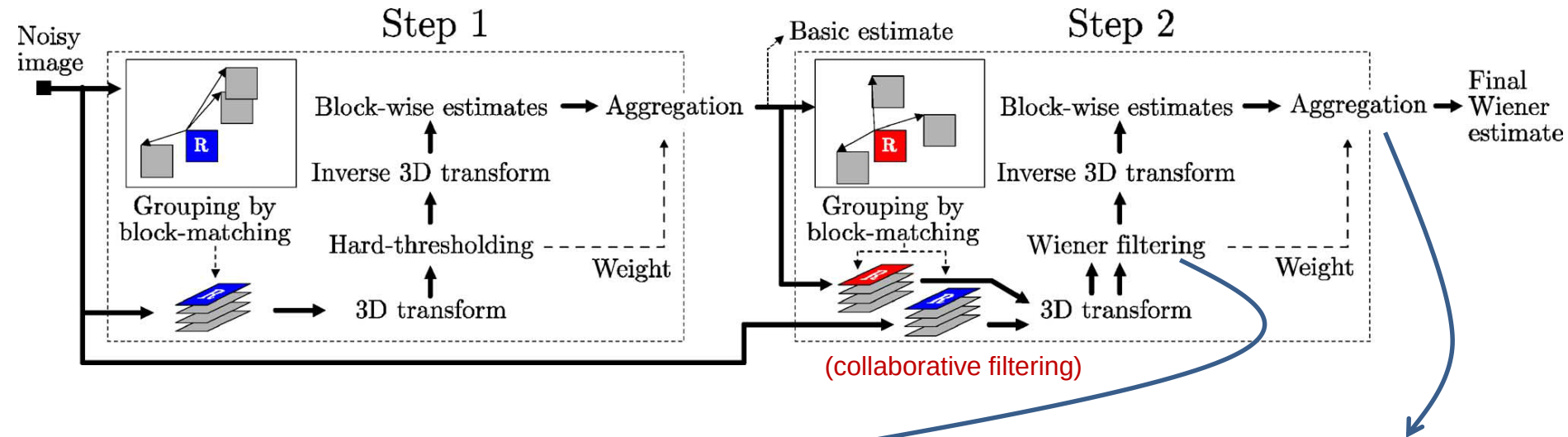
- Benchmark algorithm for state-of-the-art denoising
 - Great quality with fast implementation
 - Comprehensible, though heuristic, processing flow
- Idea
 - Block-matching for grouping similar blocks as NLM
 - Empirical coefficient-wise Wiener filtering in sparse 3-D transform domain
 - Guided by pre-filtered basic estimate

Block-matching grouping

- Predictive search used for efficient implementation



BM3D Flowchart



$$\hat{\mathbf{Y}}_{S_{x_R}^{\text{wie}}}^{\text{wie}} = \mathcal{T}_{3\text{D}}^{\text{wie}^{-1}} \left(\mathbf{W}_{S_{x_R}^{\text{wie}}} \mathcal{T}_{3\text{D}}^{\text{wie}} \left(\mathbf{Z}_{S_{x_R}^{\text{wie}}} \right) \right)$$

(Weiner filter)

$$\mathbf{W}_{S_{x_R}^{\text{wie}}} = \frac{\left| \mathcal{T}_{3\text{D}}^{\text{wie}} \left(\hat{\mathbf{Y}}_{S_{x_R}^{\text{wie}}}^{\text{basic}} \right) \right|^2}{\left| \mathcal{T}_{3\text{D}}^{\text{wie}} \left(\hat{\mathbf{Y}}_{S_{x_R}^{\text{wie}}}^{\text{basic}} \right) \right|^2 + \sigma^2}$$

(signal variance from basic estimate)

$$\hat{y}^{\text{basic}}(x) = \frac{\sum_{x_R \in X} \sum_{x_m \in S_{x_R}^{\text{ht}}} w_{x_R}^{\text{ht}} \hat{Y}_{x_m}^{\text{ht}, x_R}(x)}{\sum_{x_R \in X} \sum_{x_m \in S_{x_R}^{\text{ht}}} w_{x_R}^{\text{ht}} \chi_{x_m}(x)}$$

(aggregate **all patches** in each group)

$$w_{x_R}^{\text{wie}} = \sigma^{-2} \left\| \mathbf{W}_{S_{x_R}^{\text{wie}}} \right\|_2^{-2}$$



Heuristic parameter sets

		Fast Profile	Normal Profile	
			$\sigma \leq 40$	$\sigma > 40$
Approx. exec. time for a 256×256 grayscale image on 1.5 GHz Celeron M		0.7 sec	4.1 sec	5.8 sec
Parameters for Step 1 (<i>ht</i>)	T_{2D}^{ht}	2D-Bior1.5	2D-Bior1.5	2D-DCT
	N_1^{ht}	8	8	12
	N_2^{ht}	16	16	16
	N_{step}^{ht}	6	3	4
	N_S^{ht}	25	39	39
	N_{ES}^{ht}	6	1	1
	N_{PR}^{ht}	3	-	-
	β^{ht}	2.0	2.0	2.0
	λ_{2D}	0	0	2.0
	λ_{3D}	2.7	2.7	2.8
	τ_{match}^{ht}	2500	2500	5000
Parameters for Step 2 (<i>wie</i>)	T_{2D}^{wie}	2D-DCT	2D-DCT	2D-DCT
	N_1^{wie}	8	8	11
	N_2^{wie}	16	32	32
	N_{step}^{wie}	5	3	6
	N_S^{wie}	25	39	39
	N_{ES}^{wie}	5	1	1
	N_{PR}^{wie}	2	-	-
	τ_{match}^{wie}	400	400	3500
	β^{wie}	2.0	2.0	2.0
Common	T_{1D}	1D-Haar	1D-Haar	1D-Haar

(patch width/height)
(group element # upper bound)
(patch overlapped pixels)
(patch search range)

(transform in group direction)

BM3D denoising results

- Good quality can be achieved

($\sigma = 25$)

(a) *Lena* (PSNR 32.08 dB)



(b) *Barbara* (PSNR 30.73 dB)



(c) *Camerman* (PSNR 29.45 dB)



(d) *Man* (PSNR 29.62 dB)



(e) *Boats* (PSNR 29.91 dB)



(f) *Couple* (PSNR 29.72 dB)



BM3D denoising results

- Even applicable to seemingly hopeless noise

(Barbara, cropped)



Noisy
 $\sigma = 100$, PSNR=8.1dB



BM3D denoised
PSNR=23.5dB

Comparison of denoising results



(a) Barbara (cropped)



(b) Noisy ($\sigma = 25$)



(c) K-SVD (MSE=74.0)



(d) NLSM (MSE=61.0)



(e) BM3D (MSE=56.2)



(f) PLOW (MSE=62.6)



Extension to color-image denoising

- Use luma Y for motion estimation to group blocks
- Collaborative filtering on Y, U, V separately
- Tested RGB-to-YUV transforms:

$$A_{YCbCr} = \begin{bmatrix} 0.30 & 0.59 & 0.11 \\ -0.17 & -0.33 & 0.50 \\ 0.50 & -0.42 & -0.08 \end{bmatrix}$$

YCbCr

$$A_{\text{opp}} = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{\sqrt{6}} & 0 & \frac{-1}{\sqrt{6}} \\ \frac{1}{3\sqrt{2}} & \frac{-\sqrt{2}}{3} & \frac{1}{3\sqrt{2}} \end{bmatrix}$$

Opponent transform
(3-point DCT)

$$PSNR = 10 \log_{10} \frac{255^2}{MSE}$$

Approach to color-image denoising	<i>Lena</i>	<i>House</i>	<i>Peppers</i>
BM3D independently on the <i>R</i> , <i>G</i> , and <i>B</i> color channels	31.44	32.18	30.93
BM3D independently on each opponent color channel	32.01	32.64	31.01
C-BM3D, with grouping constraint in opponent color space	32.27	33.03	31.20



Weighted nuclear norm minimization (WNNM)

- Idea
 - Perform SVD on patch group and apply shrinkage for each component
 - Find out representative 2-D vectors and shrink singular values
 - In contrast, BM3D decomposes and shrinks in 3-D tensors
 - Shrinkage coefficients derived by WNNM formulation

$$Y = U\Sigma V^T$$

SVD on patch group

$$\hat{X} = U\mathcal{S}_\lambda(\Sigma)V^T$$

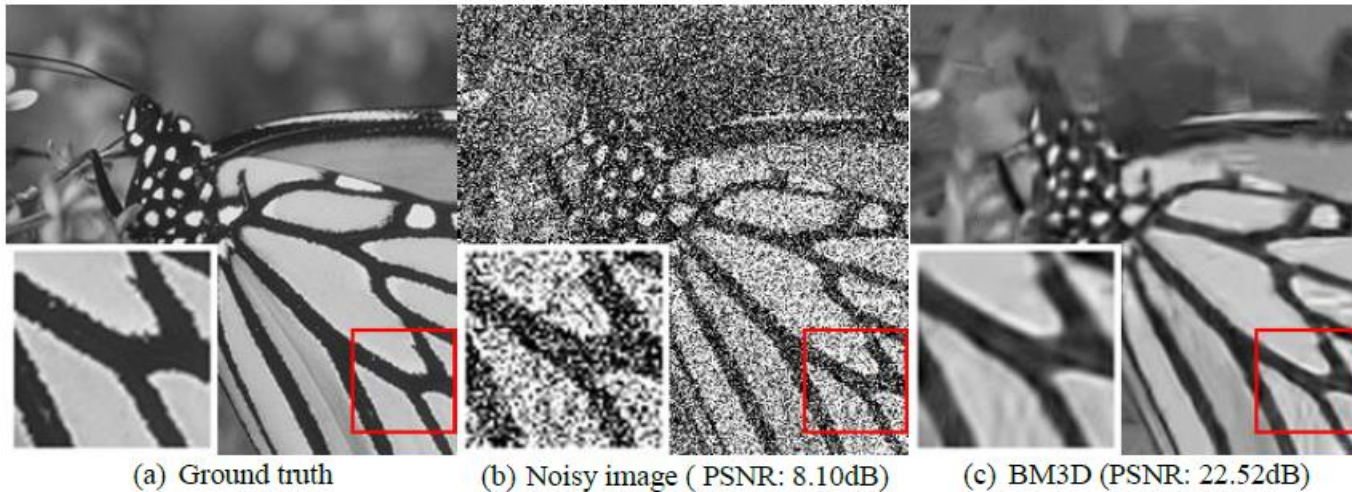
Component-wise shrinkage



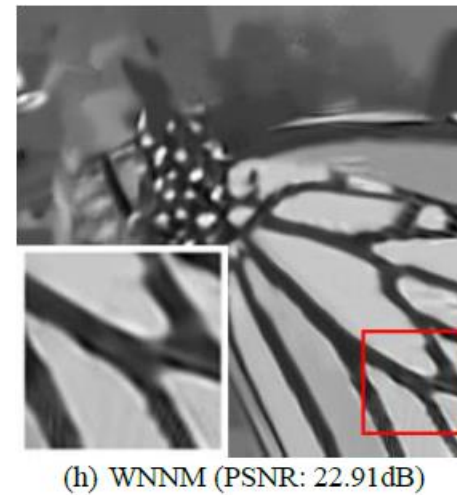
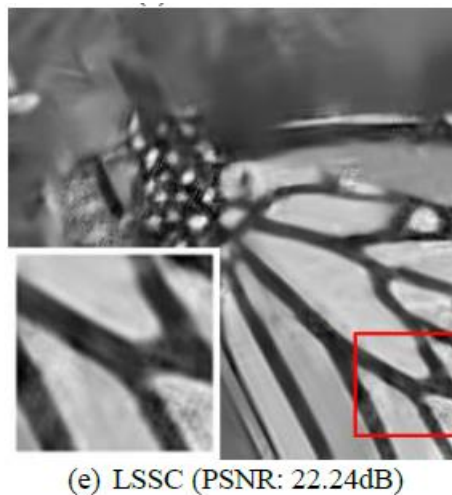
Low-rank minimization with WNNM

- Nuclear norm $\|\mathbf{X}\|_* = \sum_i |\sigma_i(\mathbf{X})|_1$
 - l^1 -norm of singular values of matrix $\mathbf{X}$
- Weighted nuclear norm (WNN) $\|\mathbf{X}\|_{\mathbf{w},*} = \sum_i |w_i \sigma_i(\mathbf{X})|_1$
 - Non-negative weights w_i
- WNNM minimization problem $\min_{\mathbf{X}} \|\mathbf{Y} - \mathbf{X}\|_F^2 + \|\mathbf{X}\|_{\mathbf{w},*}$
 - low-rank $\Rightarrow$ small WNN
- Solution of WNNM $\hat{\mathbf{X}} = \mathbf{U} \mathcal{S}_{\mathbf{w}}(\boldsymbol{\Sigma}) \mathbf{V}^T$
 - Soft-thresholding shrinkage $\mathcal{S}_{\mathbf{w}}(\boldsymbol{\Sigma})_{ii} = \max(\boldsymbol{\Sigma}_{ii} - w_i, 0)$

Denoising results



(LSSC=NLSM)



WNNM constantly outperforms BM3D by 0.3~0.4dB.
LSSC (NLSM) performs similarly to BM3D.



Complexity v.s. Quality

State-of-the-art
denoising flow

Motion
estimation

Patch
denoise

Patch
aggregation

Patch
denoising
kernel

NLM

BM3D
(3D Transform)

SVD
(SVD on patch group)

Complexity
(per patch
group)

$O(PN^2)$

$O(PN^2 \log N)$

$O(P^2N^2 + PN^4 + P^3)$

Quality

Better

Patch size: $N \times N$
Patch # in one group: P



Summary

- Categories of denoising algorithms
 - Local filter
 - Transform-based filter
 - Non-local filter
 - Sparsity-based filter
 - K-SVD, NLSM: sparse representation
 - BM3D: sparse 3-D transform + Wiener filtering
 - WNNM: low-rank matrix factorization



Appendix



Bilateral Grid

- Solve the complexity issue of BF and CBF
 - Efficient for large window size N

$$\frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix} = \frac{1}{16} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \end{bmatrix} * \begin{bmatrix} 1 \\ 4 \\ 6 \\ 4 \\ 1 \end{bmatrix}$$

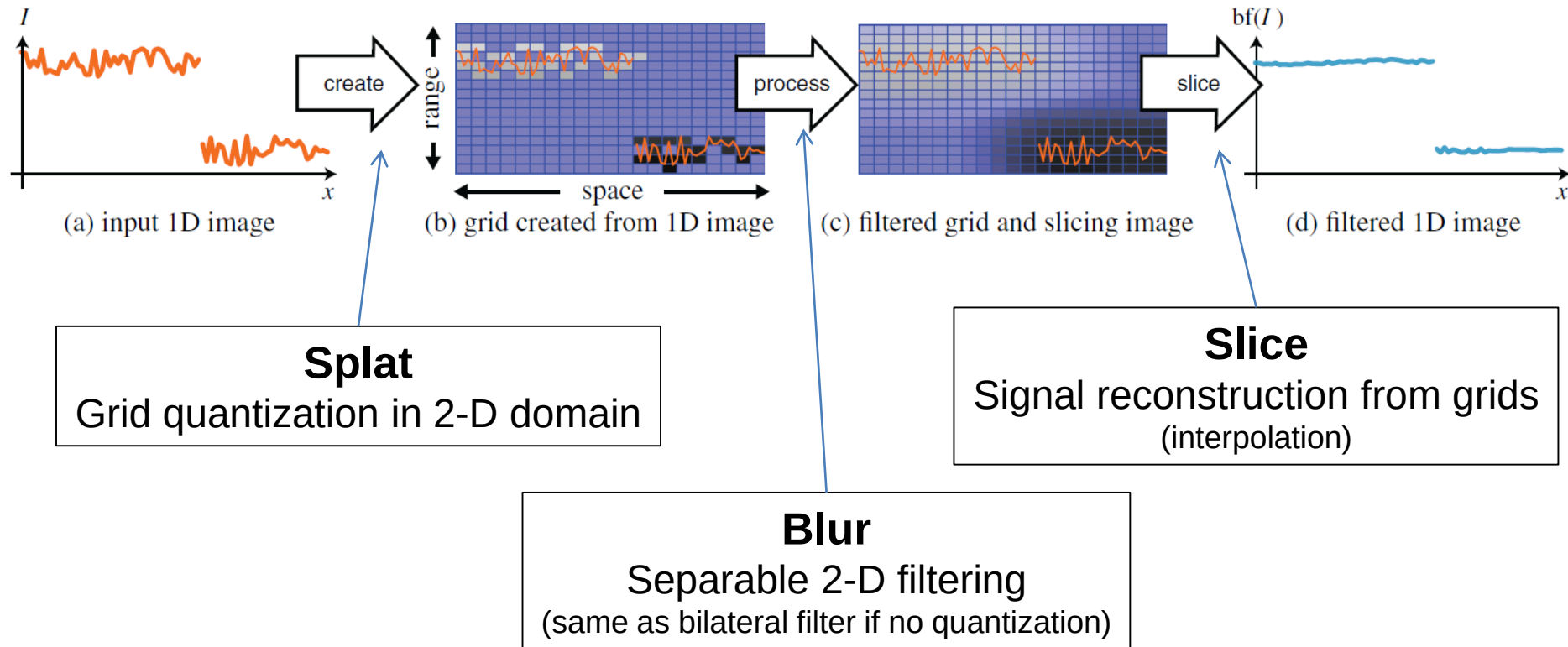
Separable filter (e.g. Gaussian)
Complexity = $O(N)$ per pixel

$$w(i, j, k, l) = \exp \left(-\frac{(i - k)^2 + (j - l)^2}{2\sigma_d^2} - \frac{\|f(i, j) - f(k, l)\|^2}{2\sigma_r^2} \right)$$

Bilateral filter
Complexity = $O(N^2)$

- **Idea:** Make non-separable bilateral filter into a *separable* filter in higher-dimension space
 - 2D image becomes a manifold in LXY domain

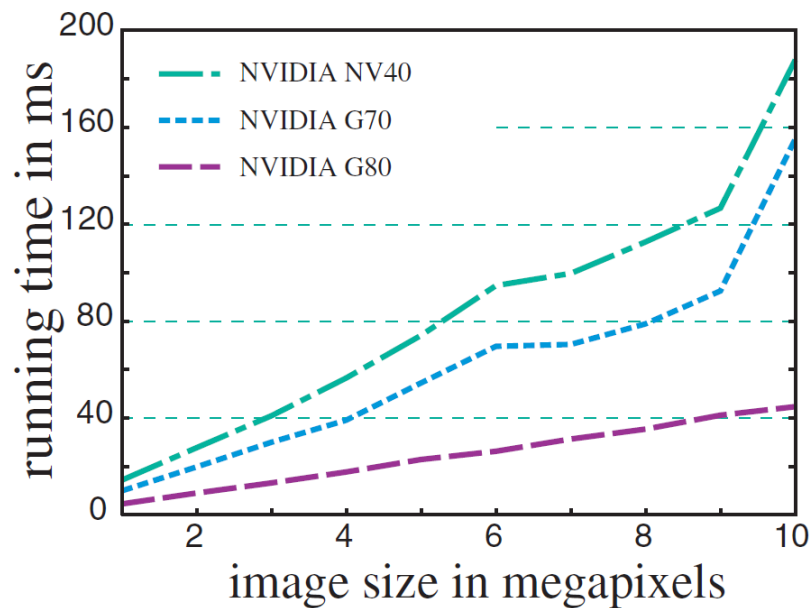
1-D Example



Complexity vs. Quality tradeoff by choosing grid size

Result

Bilateral filter
($5 \times 5 \times 5$ separable kernel in grids)



Detail enhancement



(a) input (1760 x 2140, HDR)



(b) MATLAB's adapthisteq (1.5s)



(c) our result (100ms)